Fake News Detection using Semantic Classification

Deepak TM and Umesh Goyal

1. Abstract

This report details the process and findings of developing a semantic classification model to distinguish between true and fake news articles. Leveraging the Word2Vec technique for capturing semantic relationships in text, the project aimed to build supervised learning models capable of classifying news based on meaning rather than just syntax. The methodology involved data preparation, extensive text preprocessing including lemmatization and Part-of-Speech (POS) tagging, feature extraction using a pre-trained Word2Vec model, and training/evaluation of Logistic Regression, Decision Tree, and Random Forest classifiers. Exploratory Data Analysis revealed distinct linguistic patterns between true and fake news corpora. Both Logistic Regression and Random Forest models demonstrated strong performance, achieving approximately 90% accuracy and F1-score on the validation set, indicating the viability of using semantic embeddings for fake news detection.

---

2. Problem Statement and Objective

The proliferation of digital information has led to a significant challenge in discerning credible news from misinformation, commonly termed "fake news." The ease and speed at which news spreads online necessitate automated systems to help identify and flag potentially misleading articles, thereby protecting public trust and reducing the impact of disinformation.

The primary objective of this project was to develop and evaluate a semantic classification model using the Word2Vec method to automatically classify news articles as either 'True' (Label 1) or 'Fake' (Label 0). This involved: * Preprocessing raw news text (title and body) to extract meaningful features. * Employing Word2Vec to generate semantic vector representations of the news content. * Training and comparing supervised machine learning models (Logistic Regression, Decision Tree, Random Forest) for the classification task. * Evaluating model performance using standard classification metrics.

---

3. Methodology

The project followed a structured pipeline encompassing data handling, text processing, feature engineering, and machine learning model implementation.

3.1. Data Source and Preparation

- **Datasets:** Two CSV files were used: `True.csv` (21,417 articles) and `Fake.csv` (23,502 articles).
- **Columns:** Each dataset initially contained `title`, `text`, and `date` columns.
- **Labeling:** A `news_label` column was added, assigning '1' to true news and '0' to fake news.
- **Merging:** The two datasets were concatenated into a single DataFrame.
- **Null Handling:** Rows with null values were inspected. Rows with null values in `title`, `text`, or `date` were identified. Rows missing at least three column values were dropped.

- **Text Combination:** The `title` and `text` columns were concatenated into a single `news_text` column to provide comprehensive content for analysis. The original `title`, `text`, and `date` columns were then dropped.

3.2. Text Preprocessing

A dedicated DataFrame (`df_clean`) was created for processed text. The following steps were applied sequentially to the `news_text` column:

1. **Lowercasing:** Convert all text to lowercase.
2. **Bracket Removal:** Remove text enclosed in square brackets (e.g., `[Reuters]`).
3. **Punctuation Removal:** Eliminate all punctuation marks.
4. **Number Removal:** Remove words containing numerical digits. *(This resulted in the `cleaned_text` column).*
5. **POS Tagging & Lemmatization:**
   - Utilized the `spaCy` library (`en_core_web_sm` model) for efficient processing.
   - Tokenized the `cleaned_text`.
   - Filtered tokens to keep only nouns (POS tags 'NN' and 'NNS').
   - Removed standard English stopwords.
   - Lemmatized the remaining tokens to their base form.
   - Rejoined the lemmatized nouns into a single string. *(This resulted in the `lemmatized_text` column).*
6. **Final Null Handling:** Rows where `lemmatized_text` became null after processing were dropped.

3.3. Train-Validation Split

- The processed dataset (containing `lemmatized_text` and `news_label`) was split into training (70%) and validation (30%) sets using `sklearn.model_selection.train_test_split`.
- The split was stratified based on the `news_label` to maintain the original class distribution in both sets. (`random_state=42` was used for reproducibility).

3.4. Feature Extraction (Word Embeddings)

- **Model:** The pre-trained `word2vec-google-news-300` model (from `gensim.downloader`) was used. This model provides 300-dimensional vectors for a large vocabulary of words.
- **Document Vectors:** For each news article (represented by `lemmatized_text`), a single document vector was generated by:
  1. Splitting the text into words (tokens).
  2. Retrieving the Word2Vec vector for each word present in the model's vocabulary.
  3. Calculating the mean of these word vectors.
  4. If no words from an article were found in the Word2Vec vocabulary, a zero vector of size 300 was used.
- This process was applied to both the training (`X_train_vectors`) and validation (`X_val_vectors`) sets.

3.5. Modeling Techniques

Three standard supervised classification algorithms from `sklearn` were trained and evaluated:

1. **Logistic Regression:** A linear model for binary classification (`max_iter=1000`, `random_state=42`).

2. **Decision Tree Classifier:** A tree-based model (`random_state=42`).
3. **Random Forest Classifier:** An ensemble method using multiple decision trees (`n_estimators=100`, `random_state=42`).

3.6. Tools Used

- **Data Manipulation:** Pandas, NumPy
- **Text Processing:** NLTK, spaCy, re (regular expressions), string
- **Word Embeddings:** Gensim (`word2vec-google-news-300`)
- **Machine Learning:** Scikit-learn (train_test_split, LogisticRegression, DecisionTreeClassifier, RandomForestClassifier, metrics)
- **Visualization:** Matplotlib, Seaborn, WordCloud
- **Utilities:** tqdm (progress bars)

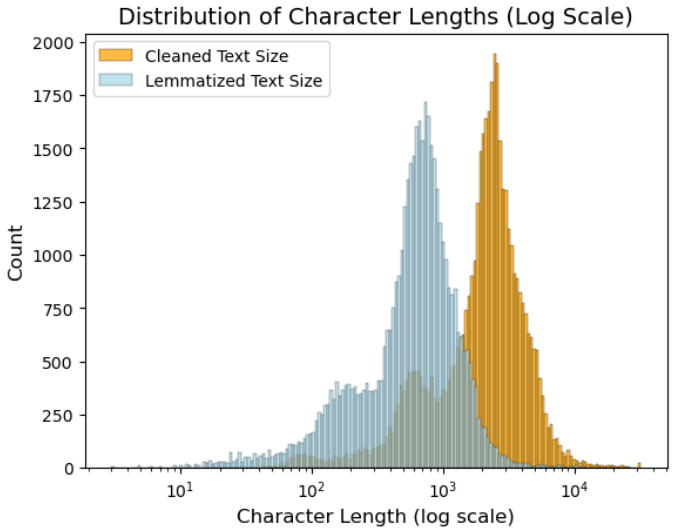
---

4. Visualizations and Analysis Results

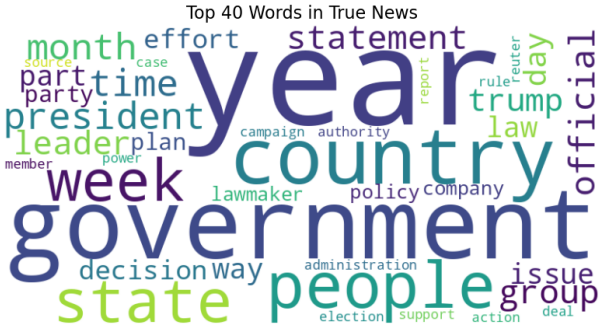
4.1. Exploratory Data Analysis (EDA) on Training Data

EDA was performed on the preprocessed training data to identify patterns and characteristics.

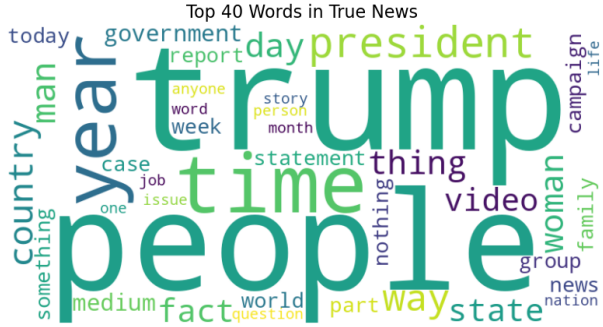
- **Character Length Distribution:** A histogram comparing the character lengths of the cleaned text and the lemmatized, POS-filtered text was plotted. The plot showed a significant reduction in text length after the stricter preprocessing, concentrating the analysis on the core nominal content. Both distributions were right-skewed.



- **Word Clouds (Top 40 Words):** Word clouds were generated for the lemmatized text of true and fake news.
  - **True News:** The word cloud highlighted terms like `trump`, `state`, `government`, `reuters`, `president`, `house`, `republican`, `people`, `year`, indicating a focus on political entities, institutions, and reporting sources.



– **Fake News:** The word cloud prominently featured `trump`, `people`, `president`, `time`, `video`, `image`, `woman`, `state`, `year`. This suggests a greater emphasis on individuals (specifically Trump), references to multimedia content (`video`, `image`), and potentially more sensational or narrative themes.



- **N-gram Frequency Analysis:** Top N-grams (unigrams, bigrams, and trigrams) were extracted and analyzed for both true and fake news.
  - **True News Unigrams:** The top unigrams were consistent with the word cloud, dominated by political and governmental terms (`trump`, `state`, `government`, `president`, `election`).
  - **True News Bigrams:** Frequent bigrams included `trump campaign`, `trump administration`, `news conference`, `request comment`, `tax reform`, reflecting common phrases in formal political reporting.
  - **True News Trigrams:** Top trigrams revealed specific reporting conventions and recurring topics, including phrases related to official statements and policy areas.
  - **Fake News Unigrams:** Similar to the true news, `trump` was the most frequent term, but other high-frequency words like `image`, `video`, and `woman` differentiated the fake news corpus.
  - **Fake News Bigrams:** Common bigrams included `trump supporter`, `president trump`, `century wire` (a likely source indicator), `police officer`, and repetitive patterns like `image image`.
  - **Fake News Trigrams:** The trigram analysis for fake news showed distinct, often repetitive phrases, some of which appear to be related to specific article templates or syndicated content (e.g., `talk radio custommade barfly philosopher...`).

These analyses collectively demonstrate clear differences in the dominant vocabulary and phraseology between true and fake news in this dataset, supporting the idea that semantic features can be discriminative.

4.2. Model Performance Evaluation

The performance of the trained models on the validation set is summarized in the following table, based on the classification reports:

Model	Accuracy	Precision (Fake News)	Recall (Fake News)	F1-Score (Fake News)	Precision (True News)	Recall (True News)	F1-Score (True News)	Weighted Avg F1
Logistic Regression	0.9006	0.91	0.90	0.90	0.89	0.90	0.90	0.90
Decision Tree	0.8229	0.82	0.84	0.83	0.82	0.80	0.81	0.82
Random Forest	0.9013	0.90	0.92	0.91	0.91	0.89	0.90	0.90

(Metrics are rounded to two decimal places)

---

5. Key Insights and Interpretations

1. **Semantic Differences are Actionable:** The EDA revealed notable differences in the subject matter and common phrases used in true versus fake news articles, even when focusing only on nouns. This provided a strong basis for using semantic embeddings.
2. **Word Embeddings Capture Discriminative Information:** Representing articles as averaged Word2Vec vectors proved effective. Despite its simplicity, this method captured sufficient semantic content for models to achieve high classification performance.
3. **Model Effectiveness:** Both Logistic Regression and Random Forest classifiers performed significantly better than the Decision Tree model. They achieved comparable high scores across accuracy, precision, recall, and F1-score (around 90%). This suggests that both linear models and ensemble methods are well-suited to the feature space created by the averaged Word2Vec embeddings.
4. **Metric Prioritization:** In the context of fake news detection, the F1-score is often a good metric to prioritize as it balances Precision (avoiding flagging true news as fake) and Recall (identifying as much fake news as possible). Both Logistic Regression (F1=0.8964 for True, 0.90 for Fake) and Random Forest (F1=0.8954 for True, 0.91 for Fake) showed strong and balanced F1-scores for both classes. The Random Forest had a slightly higher F1 for the fake class (0), which might be marginally preferred if identifying fake news is the absolute top priority, provided the precision for the true class (1) remains high, which it did (0.91).
5. **Preprocessing Impact:** The focused preprocessing pipeline, particularly the filtering to only include nouns, likely helped in creating more semantically coherent document vectors by excluding potentially noisy function words and non-noun entities.

---

6. Actionable Outcomes / Recommendations

1. **Model Selection:** Given their strong and comparable performance, either the Logistic Regression or the Random Forest model trained on Word2Vec features derived from lemmatized nouns can be selected for deployment. The Random Forest slightly edged out Logistic Regression in overall performance, particularly in identifying fake news (higher F1 for class 0), making it a potentially preferred choice.
2. **Deployment Strategy:** The chosen model can be integrated into a news platform or fact-checking tool as an initial filter to flag suspicious articles, allowing human experts to focus their efforts more efficiently on high-probability fake content.
3. **Further Feature Engineering:** Investigate alternative methods for aggregating word vectors, such as weighted averaging (e.g., inverse document frequency), or using sequence models (like LSTMs or Transformers) that can process the entire sequence of vectors rather than just averaging.

4. **Hyperparameter Tuning:** Systematic hyperparameter tuning for the Random Forest classifier could potentially yield minor improvements in performance.
5. **Source Information:** While not used in this model, incorporating metadata like the news source or author could further enhance detection accuracy.

---

7. Conclusion

This project successfully developed a semantic classification system for distinguishing true from fake news articles using Word2Vec embeddings. The methodology involved a comprehensive preprocessing pipeline focusing on key semantic units (nouns), followed by vector representation using a pre-trained Word2Vec model and training supervised classifiers.

Exploratory Data Analysis confirmed significant linguistic differences between the true and fake news corpora. The semantic features extracted through Word2Vec, combined with standard machine learning models, proved effective. Both Logistic Regression and Random Forest models demonstrated high classification performance on the validation set, achieving F1-scores around 90%, significantly outperforming the Decision Tree model. The Random Forest model showed marginally better performance in detecting fake news (class 0).

The approach effectively addressed the problem by leveraging the semantic meaning of the text, going beyond simple keyword matching, which is crucial for detecting nuanced misinformation. The relatively high performance suggests that this methodology is a promising direction for automated fake news detection.

Limitations included the simplified approach to document vectorization (simple averaging), reliance on a single pre-trained embedding model, and the restriction of text content to only nouns.

Future work could involve exploring more sophisticated embedding techniques, incorporating a wider range of linguistic features (e.g., verbs, adjectives, sentiment), experimenting with more complex model architectures (deep learning models), and evaluating the approach on diverse datasets to assess its robustness and generalizability across different domains and styles of news.